

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №2
Дисциплина: Теория параллельных алгоритмов

Работу выполнил: _____ А. А. Костров

Направление подготовки: 02.03.02 Фундаментальная информатика и
информационные технологии

Преподаватель: _____ Е. А. Нигодин

Краснодар
2026

Тема. Решение систем линейных уравнений методом Якоби.

Цель. Распараллелить процесс решения СЛАУ методом Якоби.

Задание. Составить последовательный алгоритм метода Якоби. Построить граф алгоритма. Исследовать на нём зависимости и возможности распараллеливания. Выполнить параллельную реализацию метода Якоби. Составить таблицу, отражающую сравнительное время выполнения последовательного и параллельного алгоритмов для разных размеров матриц на 2-х, 3-х и 4-х ядрах.

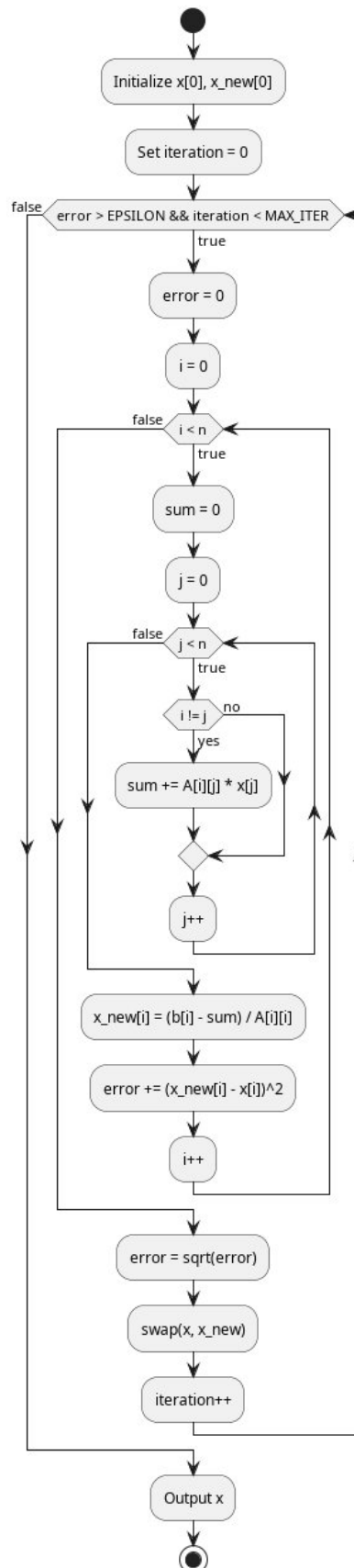
Входные данные: матрица коэффициентов A размерности $n \times n$, вектор правых частей b размерности n , начальное приближение $x^{(0)}$.

Необходимо найти: вектор решения x размерности n , удовлетворяющий системе линейных алгебраических уравнений:

$$Ax = b$$

Последовательный алгоритм можно схематично изобразить в следующем виде:

Graph of Sequential Jacobi Algorithm



Реализуем последовательный алгоритм:

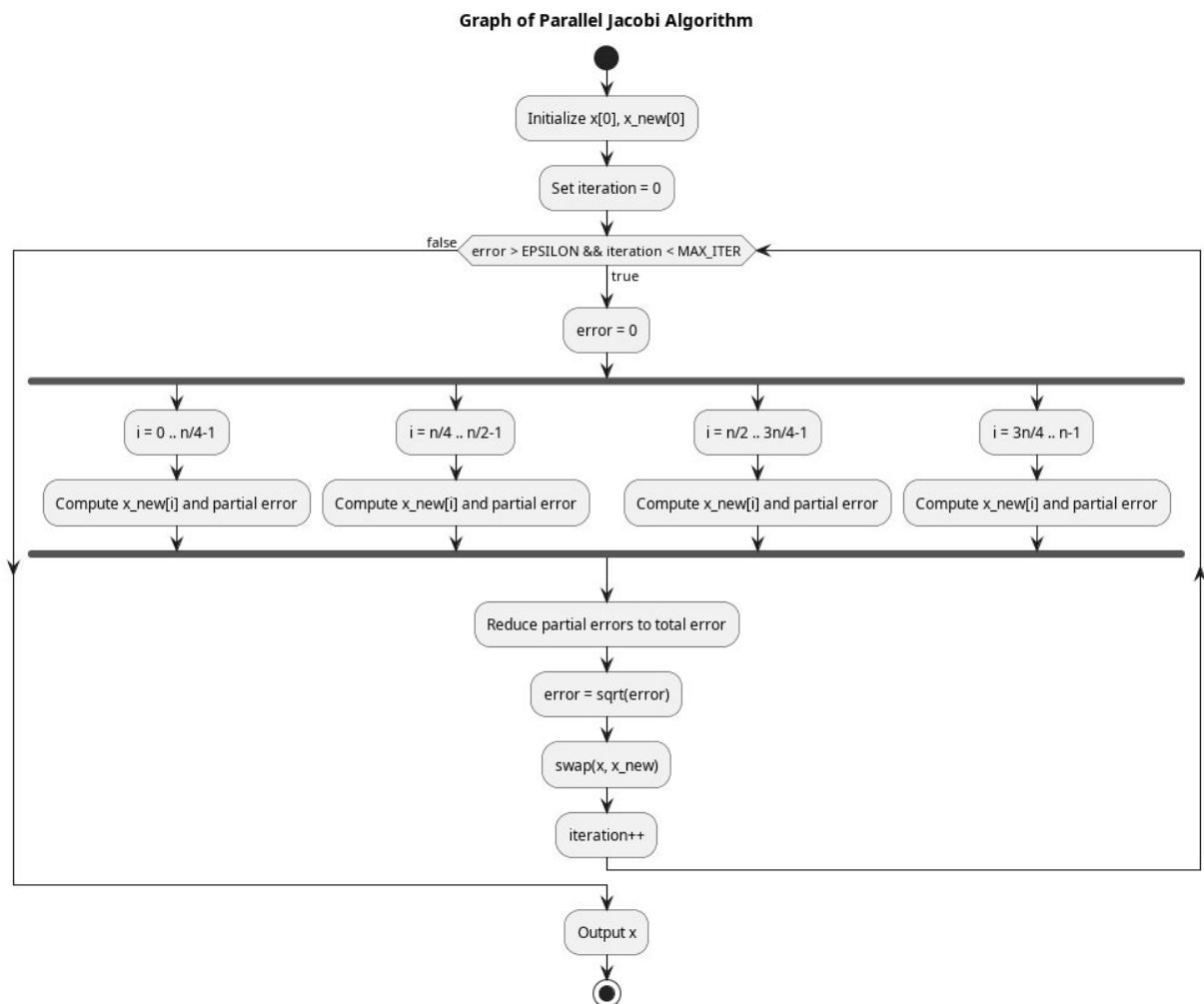
```
double sequential_jacobi(const vector<vector<double>>& A, const
vector<double>& b, vector<double>& x, int n) {
    vector<double> x_new(n, 0.0);
    int iter = 0;
    double error = 0.0;

    auto start = high_resolution_clock::now();

    do {
        error = 0.0;
        for (int i = 0; i < n; ++i) {
            double sum = 0.0;
            for (int j = 0; j < n; ++j) {
                if (i != j) {
                    sum += A[i][j] * x[j];
                }
            }
            x_new[i] = (b[i] - sum) / A[i][i];
            error += pow(x_new[i] - x[i], 2);
        }
        error = sqrt(error);
        x.swap(x_new);
        iter++;
    } while (error > EPSILON && iter < MAX_ITER);

    auto end = high_resolution_clock::now();
    return duration_cast<milliseconds>(end - start).count();
}
```

Возможность распараллеливания метода Якоби основана на том, что вычисление каждой компоненты вектора $x^{(k+1)}$ на новой итерации может выполняться независимо друг от друга, так как все они используют только значения вектора $x^{(k)}$ с предыдущей итерации. Представим систему уравнений в виде нескольких горизонтальных лент, каждая из которых содержит группу уравнений. Каждая лента параллельно обрабатывается в отдельном потоке:



Реализуем параллельный алгоритм:

```
double parallel_jacobi(const vector<vector<double>>& A, const
vector<double>& b, vector<double>& x, int n, int num_threads) {
    vector<double> x_new(n, 0.0);
    int iter = 0;
    double error = 0.0;

    omp_set_num_threads(num_threads);

    auto start = high_resolution_clock::now();

    do {
        error = 0.0;

        #pragma omp parallel for reduction(+:error)
        for (int i = 0; i < n; ++i) {
            double sum = 0.0;
            for (int j = 0; j < n; ++j) {
                if (i != j) {
                    sum += A[i][j] * x[j];
                }
            }
            x_new[i] = (b[i] - sum) / A[i][i];
            error += pow(x_new[i] - x[i], 2);
        }

        error = sqrt(error);
        x.swap(x_new);
        iter++;
    } while (error > EPSILON && iter < MAX_ITER);

    auto end = high_resolution_clock::now();
    return duration_cast<milliseconds>(end - start).count();
}
```

Полный код итоговой программы:

```
#include <iostream>
#include <vector>
#include <cmath>
#include <chrono>
#include <iomanip>
#include <omp.h>

using namespace std;
using namespace chrono;

const double EPSILON = 1e-6;
const int MAX_ITER = 1000;

void initialize_system(vector<vector<double>>& A, vector<double>& b, vector<double>& x, int n) {
    for (int i = 0; i < n; ++i) {
        double sum = 0.0;
        for (int j = 0; j < n; ++j) {
            A[i][j] = (i == j) ? n + 1 : (rand() % 10) / 10.0;
            sum += abs(A[i][j]);
        }
        b[i] = sum * (rand() % 5 + 1);
        x[i] = 0.0;
    }
}

double sequential_jacobi(const vector<vector<double>>& A, const vector<double>& b,
                        vector<double>& x, int n) {
    vector<double> x_new(n, 0.0);
    int iter = 0;
    double error = 0.0;

    auto start = high_resolution_clock::now();

    do {
        error = 0.0;
        for (int i = 0; i < n; ++i) {
            double sum = 0.0;
            for (int j = 0; j < n; ++j) {
                if (i != j) {
                    sum += A[i][j] * x[j];
                }
            }
            x_new[i] = (b[i] - sum) / A[i][i];
            error += pow(x_new[i] - x[i], 2);
        }
        error = sqrt(error);
        x.swap(x_new);
        iter++;
    } while (error > EPSILON && iter < MAX_ITER);

    auto end = high_resolution_clock::now();
    return duration_cast<milliseconds>(end - start).count();
}

double parallel_jacobi(const vector<vector<double>>& A, const vector<double>& b,
                      vector<double>& x, int n, int num_threads) {
    vector<double> x_new(n, 0.0);
    int iter = 0;
    double error = 0.0;

    omp_set_num_threads(num_threads);

    auto start = high_resolution_clock::now();

    do {
        error = 0.0;

        #pragma omp parallel for reduction(+:error)
        for (int i = 0; i < n; ++i) {
            double sum = 0.0;
            for (int j = 0; j < n; ++j) {
                if (i != j) {
                    sum += A[i][j] * x[j];
                }
            }
            x_new[i] = (b[i] - sum) / A[i][i];
            error += pow(x_new[i] - x[i], 2);
        }

        error = sqrt(error);
        x.swap(x_new);
        iter++;
    } while (error > EPSILON && iter < MAX_ITER);

    auto end = high_resolution_clock::now();
    return duration_cast<milliseconds>(end - start).count();
}

void run_benchmark(int n, int num_threads) {
```

```

vector<vector<double>> A(n, vector<double>(n));
vector<double> b(n);
vector<double> x_seq(n);
vector<double> x_par(n);

initialize_system(A, b, x_seq, n);
x_par = x_seq;

double time_seq = sequential_jacobi(A, b, x_seq, n);
double time_par = parallel_jacobi(A, b, x_par, n, num_threads);

cout << setw(10) << n << setw(15) << num_threads
    << setw(25) << fixed << setprecision(2) << time_seq << " ms"
    << setw(25) << time_par << " ms" << endl;
}

int main() {
vector<int> sizes = {256, 512, 1024, 2048, 4096};
vector<int> threads = {1, 2, 3, 4};

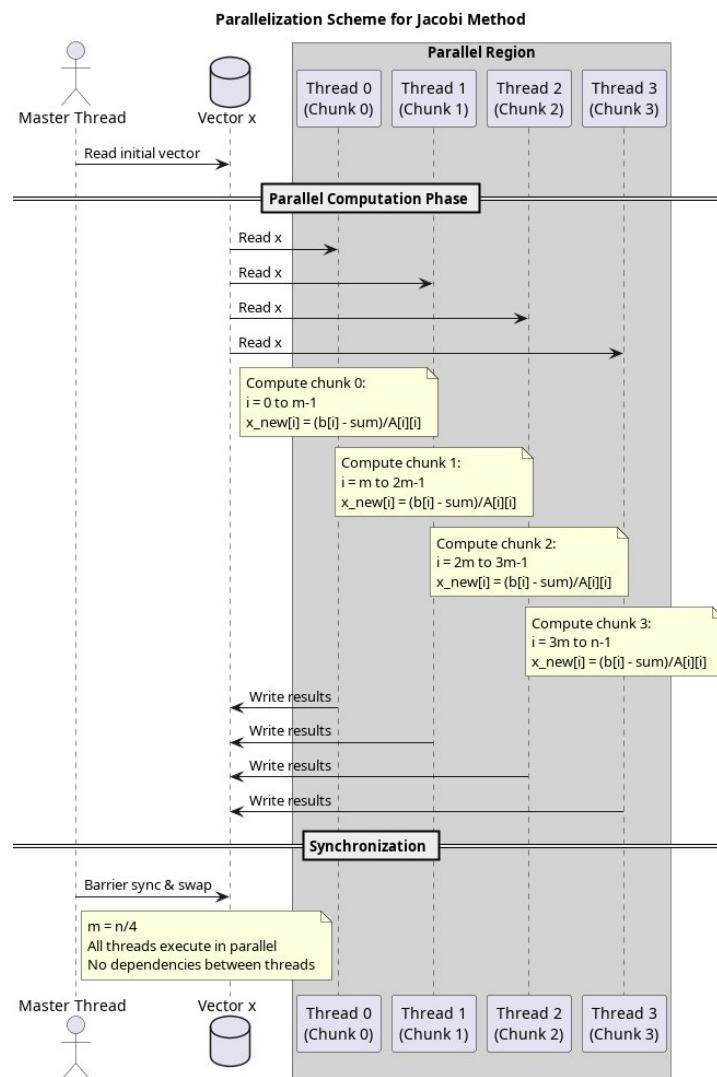
cout << "===== " << endl;
cout << setw(10) << "Size" << setw(15) << "Threads" << setw(25) << "Sequential (ms)"
    << setw(25) << "Parallel (ms)" << endl;
cout << "===== " << endl;

for (int n : sizes) {
    for (int t : threads) {
        run_benchmark(n, t);
    }
    cout << "-----" << endl;
}

return 0;
}

```

Приведём диаграмму, отражающую состояния потоков:



Зафиксируем показатели работы алгоритмов:

Size	Threads	Sequential (ms)	Parallel (ms)
256	1	14.00 ms	15.00 ms
256	2	12.00 ms	7.00 ms
256	3	14.00 ms	8.00 ms
256	4	15.00 ms	6.00 ms
512	1	46.00 ms	45.00 ms
512	2	46.00 ms	25.00 ms
512	3	48.00 ms	19.00 ms
512	4	51.00 ms	18.00 ms
1024	1	103.00 ms	83.00 ms
1024	2	115.00 ms	51.00 ms
1024	3	106.00 ms	65.00 ms
1024	4	104.00 ms	51.00 ms
2048	1	322.00 ms	320.00 ms
2048	2	320.00 ms	219.00 ms
2048	3	323.00 ms	227.00 ms
2048	4	341.00 ms	188.00 ms
4096	1	1323.00 ms	1327.00 ms
4096	2	1332.00 ms	847.00 ms
4096	3	1345.00 ms	854.00 ms
4096	4	1336.00 ms	737.00 ms

Вывод. Мы рассмотрели процесс решения систем линейных алгебраических уравнений методом Якоби, изобразили его схематично, после чего, изучив возможности распараллеливания, выполнили замеры времени для последовательного и параллельного случаев на 2-х, 3-х и 4-х потоках.